

We tried to prove two things about BDH's memory. One survived.

Building an interactive explainer for Dragon Hatchling, and what the controls kept taking away.

We set out to teach one idea: in Dragon Hatchling (BDH), a fact the model just read is held in a small, locatable set of synapses rather than in its weights. The plan had two halves. **Localisation** — ablate those synapses and that recall breaks. **Volatility** — leave them alone and enough competing facts erase the fact anyway.

Localisation survived every test we could think of. Volatility did not survive its own control. This is the account of how the second half died, because the way it died is the most useful thing we learned.

The substrate

BDH reformulates attention as a synaptic write. Its GPU formulation carries a state ρ updated by $\rho_t := (\rho_{t-1} + \text{LN}(E y_{t-1})) \times_t U$ — Eq. (8) of arXiv:2509.26507. Each token deposits a rank-1 outer product. Reading strengthens connections; answering reads them back.

We trained our own 27,776-parameter BDH-GPU on multi-query associative recall and ported the forward pass to JavaScript so it runs in the browser. The port is not trusted on assertion: the page checks its own logits against a committed Python fixture on load and shows the result in the header. If it drifts, the badge goes red and says the figures below are not trustworthy.

Two precisions that took us a while to get right, and that most summaries of BDH get wrong. ρ is $d \times n$ — in our model 32×256 — **not** neuron by neuron; the conceptual σ of Eq. (6) relates to it by $\sigma \approx D_y \rho$, rank at most d . And U is defined as "local rotation *or damping*", but the public reference implements rotation only. BDH as released has no forgetting term.

The control that carries the claim

Ablating the 8 largest-magnitude entries of the Hebbian write that laid one binding down — 0.098% of an 8,192-entry state — takes recall from 99.8% to 77.2%.

That number alone proves nothing. Removing state degrades any model. So we remove the largest-magnitude entries *outside* that write, which necessarily removes at least as much state mass — $2.06\times$ as much, in practice. Recall stays at **100.0%**.

This is the whole argument. Not "ablation hurts" but "ablation hurts *when it is aimed*, and does not when twice as much is taken from elsewhere."

We arrived at that control by failing first. Our original design matched selections on magnitude — pick untargeted entries with the same $|\rho|$ values. It kept coming in slightly *under* the targeted mass, and we eventually understood why: the entries a binding writes **are** the top of the magnitude distribution, so no equal-mass sample exists. The fix was to stop trying to match and start overshooting deliberately.

Then the second half fell over

Interference should have been symmetrical and easy. Recall of one binding against a rising number of competing bindings, alongside a control where the same number of *neutral filler tokens* is added instead. Same sequence length, same token count, differing only in whether those tokens bind anything. If only the first curve bends, forgetting is interference and not truncation.

The first version produced a clean-looking result in the wrong direction: filler degraded *earlier and harder* than competing bindings. The cause was embarrassing and instructive. Training covered 2–8 bindings but only 0–4 filler units. Each curve fell shortly after leaving *its own* training range. We were measuring distance-outside-distribution, not mechanism. Inside the shared range both sat at 100%, because a state of 32,768 entries is never stressed by 16 bindings.

So we rebuilt it: matched training coverage across both conditions, and a state narrow enough that bindings actually compete. That version gave the dissociation we wanted — recall collapsing $99.6\% \rightarrow 7.2\%$ against competing bindings while filler stayed flat at 100%.

And then we ran one more check, and it failed.

Hold the number of competing bindings fixed — 7 bindings, always querying the oldest — **and vary only the neutral filler**. Recall reads 50.4%, 40.0%, 73.6%, 100.0% across filler 0 to 3. The binding count never moved. Recall moved 50 points. The "load" axis was confounded with sequence geometry, and every interference curve we had built on it was uninterpretable.

We withdrew the result. It took half the claim with it.

What was actually going on

Chasing that confound produced the best finding in the project.

Scanning recall across bindings $\times$ query position $\times$ filler — 140 cells — showed that retrieval fails in **periodic bands of relative offset**. Mean recall by token distance from query back to binding: 100% at offset 1, **42.8% at offset 3**, 99.7% at 5, 97–99% through 11, then a second null band at 13–17, then recovery. With no filler, the binding sitting exactly 3 tokens before the query scores **0.0%** — not degraded, zero — in all seven cells from 2 to 8 bindings, while its neighbours sit at 98–100%.

We got this wrong twice on the way. First we declared a clean offset-3 rule. Then we refuted it, because adding filler made the dip vanish — except the dip vanished only because at two filler units *no binding can occupy offset 3 any more*; the index at offset 3 is $P + f - 2$, which stops existing. Filler never repaired the model, it slid the blind spot off the end of the list. Then we over-corrected the other way and claimed the offset rule predicted every failure, which our own committed data contradicts: 18 of 27 failing cells are not at offset 3.

The honest version is narrower than any of those three: two null bands, one exact sub-rule inside the first, and a mechanism (RoPE phase cancellation at specific relative offsets) that fits everything and that we have not demonstrated.

What we would tell someone starting this

A control is not a formality. Every result we lost, we lost to a control, and each one was cheap to run and would have been expensive to skip. The interference curve looked *better* before we checked it.

Matched conditions means matched in every respect, including how much of each the model was trained on. We made this mistake twice — unmatched training coverage between two conditions, then unmatched training quality between four model widths — before we recognised it as one mistake wearing two costumes.

An aggregate effect does not license a per-case prediction. Our own test asserted that targeted ablation lowers $p(\text{answer})$ on a given sequence. It failed, and it was the test that was wrong: at this dose the answer flips on about 23% of sequences. That mattered for the artifact too — a page opening on a random draw would show nothing breaking three times in four. So the opening sequence is chosen by a stated rule and the page says so, and "new sequence" always draws unselected. The learner meets the probabilistic nature of the effect in the first ten seconds instead of being shown a best case dressed as typical.

Publish the failures on the page, not just in the repo. Three results are on our explainer under "What we did not get": decay, which never produced the stability–plasticity crossover it predicts; a capacity ordering confounded by training quality; and the interference curve above. A reader can check what we did not manage. We think that is worth more than a fourth result would have been.

Where this connects

BDH-CQ carries the same principle one level up. Its contextual memory S_t is updated by demonstrations at inference; nothing in the weights changes. Its published interventions say what our ablation says: with byte-identical held-out inputs, one demonstration at the right complexity lifts depth-five nesting from 19/24 to 24/24, and recovers 13/24 from a 0/24 baseline at ordering length eight. What is in state decides.

Label that evidence carefully, because it is easy to overstate. The 150M configuration reaches 29.5% pass@2 on public ARC-AGI-1 at a computed \$0.00070/task — a **cost-efficiency** result, not an accuracy win, since GPT-5.6 Luna (Low) scores 34.2% on the same set. The audit reproducing 29.5% was run by co-authors, so it is not an external reproduction. And capability there is structured rather than scalar: rotation composes with relocation on 72/72 tasks, colour swap on 0/72.

Try it: <<https://ritik650.github.io/DataForge-Pathway/>> — the model runs in your browser at about 2 ms a forward pass. Click a condition, drag the dose, and press *run 100 trials* to put your own number beside ours.

Source, data and the full record of what we withdrew: <<https://github.com/Ritik650/DataForge-Pathway>>

Our model is our own small training run of the published BDH-GPU architecture on a synthetic task — not an official Pathway model, not a released checkpoint, and far below any reported BDH scale. It is evidence about the mechanism, not a reproduction of Pathway's results.